DEEP LEARNING INDABAX Uganda

Sunbird AI

MLOps Pipelines

Data Collection to Model Deployment

IndabaX Uganda 2026 | **Tibabwetiza Muhanguzi Joel** | Embedded & Machine Learning Engineer - Sunbird AI

PyTorch | MobileNetV2 | W&B | Streamlit | Hugging Face | ONNX/TFLite

What is "Ops"?

From DevOps to MLOps — engineering discipline applied to machine learning

Where "Ops" comes from

DevOps emerged in software engineering to bridge the gap between developers who write code and operations teams who run it in production. The insight was simple: building software and running software reliably are equally important. MLOps applies that same thinking to machine learning — training a model and keeping it working reliably in production are both part of the job.

🔁 Reproducibility

Can you recreate exactly what produced this model? Same data, same code, same hyperparameters. Experiment tracking with W&B solves this.

🚀 Deployment

Can you reliably serve the model to users? Streamlit, FastAPI, Docker and cloud platforms solve packaging and delivery.

👁️ Monitoring

Is the model still working after deployment? Logging predictions, tracking confidence, and detecting drift keeps it honest over time.

"MLOps is what happens between 'my model works on my laptop' and 'my model is reliably helping real users in production'"

Plain ML vs MLOps

Why is MLOps necessary in practice?

Plain ML		MLOps
• One-off notebook experiment • model.pkl on your laptop • Works on my machine • No monitoring after deployment • Retrain? Manual, ad-hoc	vs	• Reproducible versioned runs • Model registry with lineage • Portable artifacts + env spec • Drift detection & alerts • Automated retrain pipeline

The MLOps Pipeline

8 stages from raw data to monitored, edge-ready model

↺ ***Retrain trigger when drift detected***

The Dataset

Breast Ultrasound Images — Shivam Rai sharma/breast cancer dataset on Hugging Face

780	600	500px	3
Total Images	Female Patients	Avg Image Size	Classes

Normal	Benign	Malignant
Healthy tissue — no abnormality detected	Non-cancerous mass — monitor over time	Cancerous mass — requires urgent attention

Data Collection & Its Complexities

The quality of your model is bounded by the quality of your data

What Data Collection Involves

- Identifying relevant data sources
- Defining collection protocols & consent
- Storage, formatting & versioning
- Labelling / annotation by domain experts
- Ensuring class balance & diversity
- Continuous collection as context changes

Data Scarcity

Medical datasets like ours have only 780 images — far less than the tens of thousands ideal for deep learning, urban noise uganda 61k dataset?

Label Quality

Labels depend on expert clinicians. Inter-annotator disagreement is common and hard to resolve at scale.

Privacy & Ethics

Patient data requires consent, anonymisation, and compliance with regulatory authorities.

Distribution Shift

Data collected in 2018 on one machine may not represent images from a different hospital or scanner in 2026.

Tools & Stack

Everything used in this pipeline is free and open source

Hugging Face Datasets

Data source

Load breast ultrasound images in one line

PyTorch + torchvision

Model training

MobileNetV2 fine-tuning with transfer learning

Weights & Biases

Experiment tracking

Log params, metrics, artifacts — live dashboard

Streamlit

Web deployment

Upload image ⟶ get prediction in browser

Median.co

Mobile APK

Wrap Streamlit URL as Android app in 5 min

ONNX / TFLite

Edge deployment

Quantized model for Raspberry Pi & edge devices

Model: Transfer Learning

MobileNetV2 pre trained on ImageNet — fine-tuned for 3-class ultrasound classification

Why Transfer Learning?

- Only ~780 images available
- Training from scratch needs 10,000+
- ImageNet features generalise well
- Good accuracy in 3–10 epochs
- 3.4M params vs ResNet-18's 11M

Architecture Flow

Input (grayscale)	Conv2d (adapted)	MobileNetV2 backbone	Classifier (3 classes)	Softmax output

Key Adaptations

- Conv1: 3 channels → 1 (grayscale)
- Final FC: ImageNet 1000 → 3 classes
- Trained on Colab T4 GPU in ~3 min

Experiment Tracking

Weights & Biases — every run is logged, compared, and versioned

Parameters

Hyperparameters set before training: lr, epochs, batch_size, model type

Metrics

Scores logged per epoch: train_loss, val_loss, val_acc, test_acc

Artifacts

Saved files: model weights (.pt), classification reports

Run

One training execution — unique ID, timestamp, full lineage

```
wandb.log({ "val_loss": 0.42, "val_acc": 0.87 })   ← runs every epoch, appears live on wandb.ai
```

Deployment

From saved weights to browser, mobile, and edge

1. **Save weights**
   torch.save(model.state_dict(), 'mobilenetv2_ultrasound.pt')

2. **Build Streamlit app**
   Upload image → preprocess → load model → predict → show result

3. **Deploy to cloud**
   Push to GitHub → connect Streamlit Community Cloud → live URL

4. **Wrap as mobile APK**
   Paste URL into Median.co → enable file permissions → download APK

5. **Edge / Quantize**
   Export to ONNX or TFLite → deploy on Raspberry Pi or MCU

Edge Deployment & Quantization

Run your model on Raspberry Pi — no internet, no cloud required

Why Quantize?

- Float32 → Int8 reduces model size by ~4x
- Faster inference on CPU-only devices
- Works offline — no network dependency
- Raspberry Pi 3B+ can run MobileNetV2 Int8
- Minimal accuracy drop (<2% typical)

ONNX

Universal format — works with OpenCV, ONNX Runtime on Pi

TFLite

TensorFlow Lite — optimised for ARM, used in SunEcho pipeline

OpenVINO

Intel optimised — great for x86 edge devices

```
torch.onnx.export(model, dummy, 'model.onnx')
```

Production Monitoring

Deployment is the beginning — not the end

Prediction Distribution

Are outputs shifting? Logging every inference to W&B reveals trends over time.

Confidence Scores

Low confidence = model seeing something unfamiliar. Alert if avg drops below 60%.

Data Drift (PSI)

Input features changing from training distribution? Population Stability Index detects this early.

Ground Truth Feedback

Users confirm or correct predictions → accumulate real-world labelled data for retraining.

What We Built Today

📓 Colab notebook — full pipeline from raw images to trained model

📊 W&B dashboard — live training curves and model artifact registry

🌐 Streamlit app — upload image, get prediction, live at a public URL

📱 Android APK — the same app wrapped for mobile via Median.co

⚡ Edge-ready — ONNX/TFLite export for Raspberry Pi deployment

Thank you · Questions?